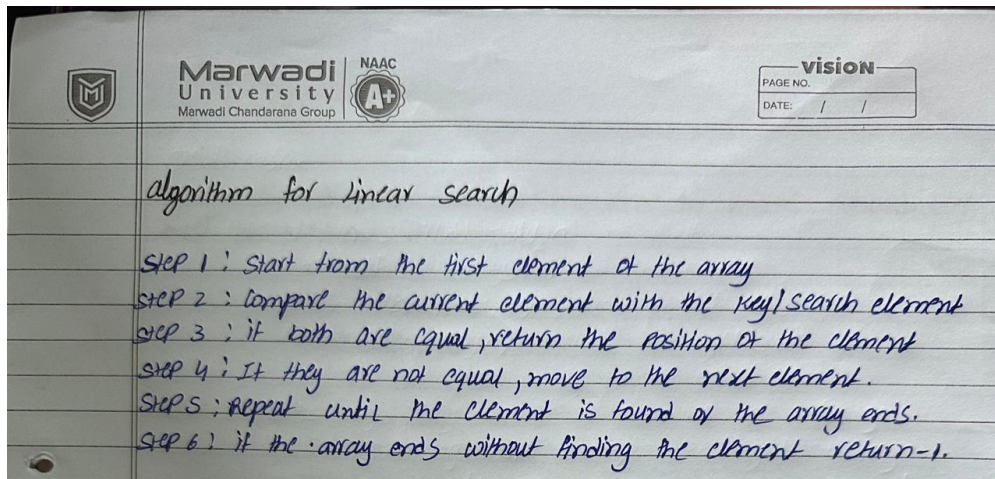


PRACTICAL - 2

AIM : Implementation and Time analysis of Linear and Binary search algorithm.

ALGORITHM // Linear search



PROGRAM : // Linear search

```
#include <iostream>
using namespace std;

int main() {
    int n, key;

    cout << "Enter the number of elements: ";
    cin >> n;

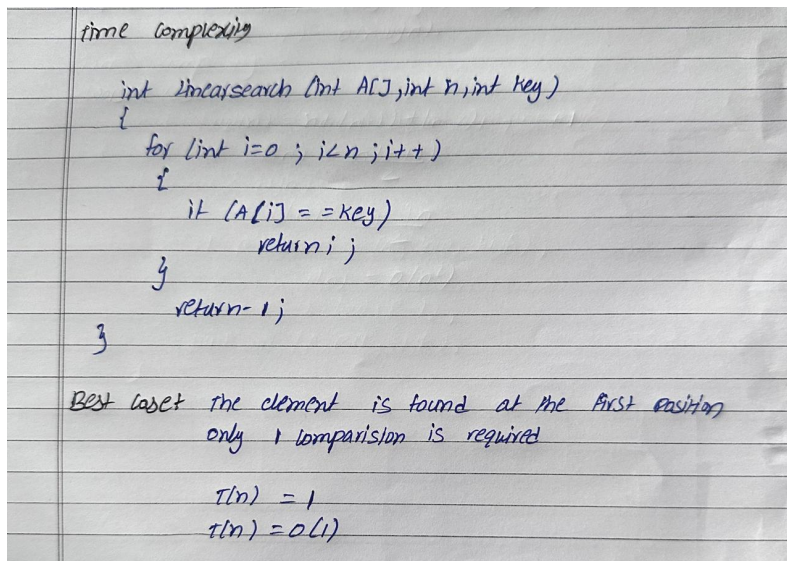
    int arr[n];

    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    cout << "Enter the element to search: ";
    cin >> key;
    int position = -1;
```

```
for (int i = 0; i < n; i++) {  
    if (arr[i] == key) {  
        position = i;  
        break;  
    }  
}  
if (position != -1) {  
    cout << "Element found at index: " << position << endl;  
} else {  
    cout << "Element not found." << endl;  
}  
return 0;  
}
```

OUTPUT :

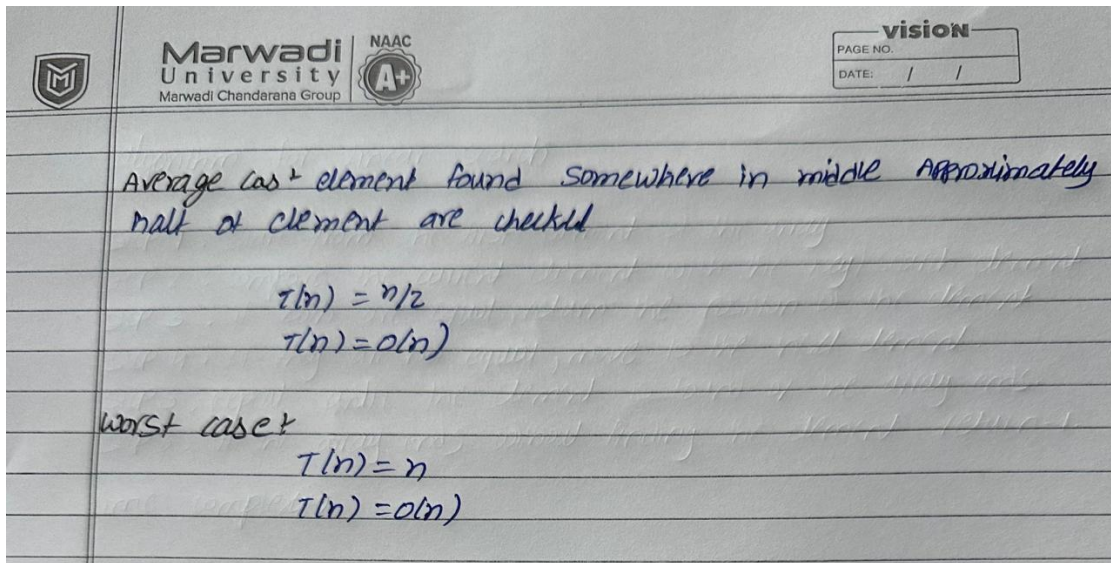
```
Enter the number of elements: 4  
Enter 4 elements: 12 24 0 1  
Enter the element to search: 0  
Element found at index: 2  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

TIME COMPLEXITY : // Linear search

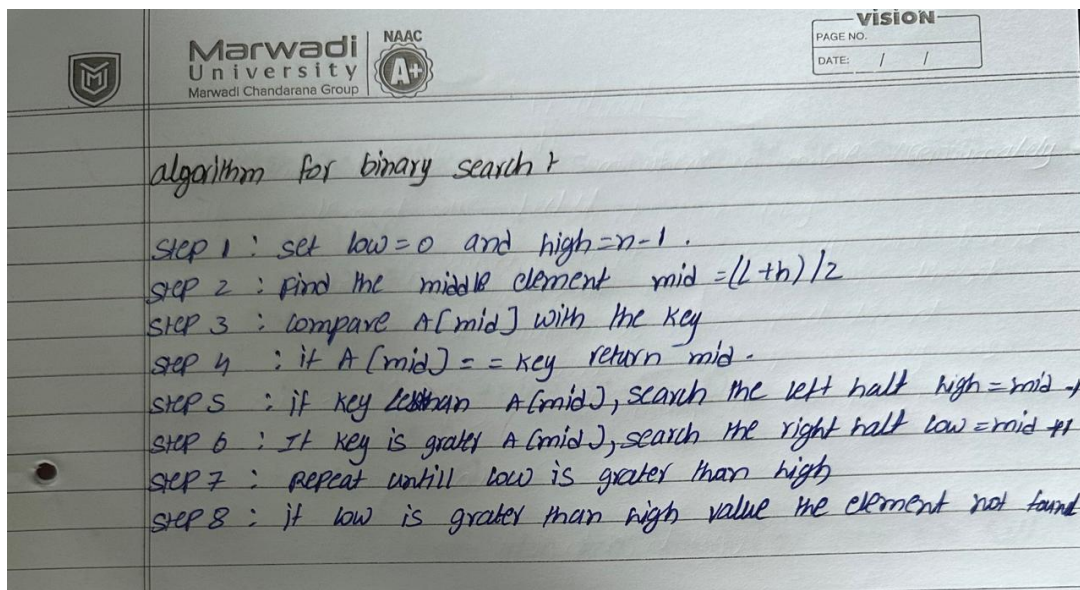
```
time complexity  
int linearsearch (int A[],int n,int key)  
{  
    for (int i=0 ; i<n ; i++)  
    {  
        if (A[i] == key)  
            return i ;  
    }  
    return -1 ;  
}
```

Best case the element is found at the first position
only 1 comparison is required

$$T(n) = 1$$
$$t(n) = O(1)$$



ALGORITHM // Binary search



PROGRAM : // Binary search

```
#include <iostream>
using namespace std;
int main()
{
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
```

```
int arr[n];
cout << "Enter " << n << " elements:\n";
for (int i = 0; i < n; i++) {
    cin >> arr[i];
}

int target;
cout << "Enter the target element to search: ";
cin >> target;

// Binary Search Algorithm

int low = 0;
int high = n - 1;
bool found = false;
while (low <= high) {
    int mid = low + (high - low) / 2;

    if (arr[mid] == target) {
        found = true;
        cout << "Element found at index: " << mid << endl;
        break;
    }
    else if (arr[mid] < target) {
        low = mid + 1;
    }
    else {
        high = mid - 1;
    }
}
}
```

OUTPUT :

```
Enter the number of elements: 4
Enter 4 elements : 1 0 4 8
Enter the target element to search: 4
Element found at index: 2

...Program finished with exit code 0
Press ENTER to exit console.
```



TIME COMPLEXITY : // Binary search

Time complexity :

```
low = 0
high = n - 1
while (low <= high) {
    mid = (low + high) / 2
    if (A[mid] == key)
        return mid
    else if (A[mid] < key)
        low = mid + 1
    else
        high = mid - 1
}
```

Marwadi University
Marwadi Chandarana Group

NAAC
A+

vision
PAGE NO. _____
DATE: / /

Best case the element found mid in the first comparison

$$T(n) = 1$$
$$T(n) = O(1)$$

Worst case :

$$T(n) = n + n/2 + n/4 + n/8 + \dots + 1$$
$$T(n) = \log_2 n$$
$$T(n) = O(\log n)$$

Average case :

The search also keeps dividing the array into half.

$$T(n) = n + n/2 + n/4 + n/8 + \dots + 1$$
$$T(n) = O(\log n)$$